

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	K. Srinivasa Reddy	Reg. No.:	192525452
Section / Batch:	AIMIL	Date:	17/07/2026

Code of Conduct

I K. Srinivasa Reddy (192525452) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student:
K. Srinivasa Reddy

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

Print Odd and Even Counts in a Singly Linked List

Given

77, 98, 22, 11, 66, 27, 55, 65, 19, 10

Understanding of the Problem

The given singly linked list contains 10 elements. We have to traverse the list and find the number of odd and even elements.

* If $\text{data} \% 2 == 0$, the element is even

* Otherwise the element is odd

Appropriate Data Structure

A singly linked list is used. Each node contains.

Data $\rightarrow$ Next

Algorithm

1. Set $\text{odd} = 0$ and $\text{even} = 0$
2. Set $\text{temp} = \text{head}$
3. Traverse the list until $\text{temp} = \text{NULL}$.
4. If $\text{temp} \rightarrow \text{data} \% 2 == 0$, increment even.
5. Otherwise, increment odd.
6. Move temp to the next node.
7. Print the odd and even counts.

Code

```
#include <stdio.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node * next;
```

```
};
```

```

int main() {
    int a[] = {77, 88, 22, 11, 66, 27, 55, 65, 18, 109};
    int odd = 0, even = 0;
    for (int i = 0; i < 10; i++) {
        if (a[i] % 2 == 0)
            even++;
        else
            odd++;
    }
    printf("odd count = %.d\n", odd);
    printf("Even count = %.d\n", even);
    return 0;
}

```

Out put:

odd count = 5

Even count = 5

complexity Analysis

Time complexity : $O(n)$

Space complexity : $O(1)$

Result

odd count = 5

Even count = 5

2 Linear Search for $k = 20$ in a Singly Linked List
Given

77, 88, 22, 11, 66, 27, 55, 65, 18, 10, ,

Key : $k = 20$

Understanding of the problem

We have to search for the key element 20 in the Singly linked list using linear search.

* If 20 is found $\rightarrow$ Print Yes

* if 20 is not found $\rightarrow$ Print No

Appropriate Data Structure

A Singly Linked list is used. we start from the first node and visit each node sequentially.

Algorithm

1. Set $key = 20$
2. Set $temp = head$
3. Compare $temp \rightarrow data$ with key.
4. IF equal, Print Yes and stop.
4. Otherwise, move to the next node.
6. Continue until Null.
7. IF the key is not found, print No.

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
int a[] = {77, 88, 22, 11, 66, 27, 55, 65, 18, 10};
```

```
int key = 20, found = 0;
```

```
for (int i = 0; i < 10; i++) {
```

```
if (a[i] == key) {
```

```
found = 1;
```

```
break;
```

}

}

if (found)

printf("yes");

else

printf("No");

return 0;

}

Output:

No

Complexity Analysis

* Best Case : $O(1)$

* Worst case : $O(n)$

* Space Complexity : $O(1)$

Result

~~$k = 20$~~

Output : No

~~Output~~

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, wellstructured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear
Faculty In-charge		Course Coordinator		Program Director	
Signature: 		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	